Reviewer Pack — Minimal Rank of Generalized Theta Functions and Resolution P…

Entry da623503e2fc · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-06-07 14:59:38 UTC

Minimal Rank of Generalized Theta Functions and Resolution Proof Width

Entry ID: `da623503e2fc` **Verdict**: `INCONCLUSIVE` **Recorded**: 2026-06-07 14:59:38 UTC

1. Conjecture

Field A (mathematical branch): Theta Function Theory **Field B** (complexity object): Boolean Satisfiability (Resolution Proof Complexity)

Statement:

> For every CNF φ, the minimal rank of the generalized theta function associated with φ is linearly correlated with its resolution proof width, such that θ_min(φ) = Θ(w(φ)) for all instances with at most n variables.

Rationale (proposer's reasoning):

> Theta functions are a classical topic in mathematics with connections to number theory and geometry. If their ranks could be related to the complexity of resolution proofs, it might reveal new insights into the structure of SAT problems.

Taxonomy category: `Theta_Funcs` (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): `d720b7d348d3e8ff`

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

> The conjecture is supported if, for all CNFs φ with at most n variables, the correlation coefficient between θ_min(φ) and w(φ) is ≥ 0.95, where θ_min(φ) is the minimal rank of the generalized theta function and w(φ) is the resolution proof width.

3. Barrier filter (F1)

Final: `PASS`

Barrier	Verdict	Confidence	LLM_1	LLM_2
RELATIVIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN

Barrier	Verdict	Confidence	LLM₁	LLM₂
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	1.00	SAFE	UNCERTAIN

4. Novelty audit

Verdict: `ADJACENT_OK` against 8 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - `theta functions AND resolution proof complexity` - `generalized theta functions AND CNF minimal rank` - `resolution proof width IN BOOLEAN Satisfiability`

Top relevant hits considered: - [http://arxiv.org/abs/1511.03716v2] Evaluations of certain theta functions in Ramanujan theory of alternative modular bases - [http://arxiv.org/abs/1808.07970v6] Properties of Lerch Sums and Ramanujan's Mock Theta Functions - [http://arxiv.org/abs/1407.6169v3] Multiplicative Complexity of Vector Valued Boolean Functions - [http://arxiv.org/abs/math/0305034v2] A canonical decomposition of generalized theta functions on the moduli stack of Gieseker vector bundles - [http://arxiv.org/abs/hep-th/0107222v2] Minimal representations, spherical vectors, and exceptional theta series I - [http://arxiv.org/abs/1103.5740v2] Generating and Searching Families of FFT Algorithms - [http://arxiv.org/abs/2206.00903v2] Satisfiability of Quantified Boolean Announcements - [http://arxiv.org/abs/astro-ph/0407622v3] A Study of the Composition of Ultra High Energy Cosmic Rays Using the High Resolution Fly's Eye

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)
Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution**: rc=124, elapsed=249.4s

5.1 Generated Python source

```python
import random
import math
from fractions import Fraction

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_cnf(n):
        clauses = []
        for _ in range(2**n):
            clause = [random.randint(-n, -1), random.randint(1, n)]
            if random.choice([True, False]):
                clause.reverse()
            clauses.append(clause)
        return clauses

    def resolution_width(cnf):
        queue = set()
        for clause in cnf:
            queue.add(clause[0])

        while True:
            new_queue = set()
            for literal in queue:
                if literal > 0:
```

```
                continue
            neg_literal = -literal
            for clause in cnf:
                if neg_literal in clause and len(clause) == 2:
                    new_clause = [l for l in clause if l != neg_literal]
                    if new_clause not in new_queue:
                        new_queue.add(new_clause)
        if new_queue == queue:
            break
        queue.update(new_queue)

    return max(len(cnf), len(queue))

def theta_min(cnf):
    n = len(cnf[0])
    matrix = [[0] * (n + 1) for _ in range(n + 1)]
    for clause in cnf:
        for literal in clause:
            if literal > 0:
                row, col = literal - 1, n
            else:
                row, col = -literal - 1, n
            matrix[row][col] += 1

    # Gaussian elimination to find rank
    def gaussian_elimination(matrix):
        rows, cols = len(matrix), len(matrix[0])
        rank = 0
        for i in range(cols):
            pivot_row = None
            for j in range(rank, rows):
                if matrix[j][i] != 0:
                    pivot_row = j
                    break
            if pivot_row is None:
                continue
            matrix[pivot_row], matrix[rank] = matrix[rank], matrix[pivot_row]
            rank += 1
            for j in range(rank, rows):
                factor = -matrix[j][i] / matrix[rank-1][i]
                for k in range(i, cols):
                    matrix[j][k] += factor * matrix[rank-1][k]
        return rank

    return gaussian_elimination(matrix)

n_max = 40
instances_tested = 30
total_theta_min = 0
total_width = 0

for _ in range(instances_tested):
    cnf = generate_cnf(random.randint(5, n_max))
    theta_min_value = theta_min(cnf)
    width = resolution_width(cnf)
```

```
        total_theta_min += theta_min_value
        total_width += width

    mean_theta_min = total_theta_min / instances_tested
    mean_width = total_width / instances_tested

    correlation_coefficient = (instances_tested * sum(theta_min_value * width for theta_min_value,

    conjecture_holds = correlation_coefficient >= 0.95
    counterexample = "" if conjecture_holds else "mapping_undefined"

    return {
        "metric_name": "correlation_coefficient",
        "metric_value": correlation_coefficient,
        "instances_tested": instances_tested,
        "n_max": n_max,
        "conjecture_holds": conjecture_holds,
        "counterexample": counterexample
    }

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29] * 3

    results = []

    for seed in seeds:
        trial_result = run_trial(seed)
        print(f"TRIAL: {trial_result}")
        results.append(trial_result)

    mean_value = sum(result["metric_value"] for result in results) / len(results)
    std_value = math.sqrt(sum((result["metric_value"] - mean_value)**2 for result in results) / le
    support_fraction = sum(1 for result in results if result["conjecture_holds"]) / len(results)

    if all(result["conjecture_holds"] for result in results):
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_frac
    elif support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_frac
    else:
        first_failing_seed = next(seed for seed, result in zip(seeds, results) if not result["conj
        print(f"RESULT: FALSIFIED counterexample=\"mapping_undefined\" first_failing_seed={first_f
```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
TIMEOUT after 240s
```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: `INCONCLUSIVE`

Reasoning:

> The test timed out before producing data, which means the pre-registered support condition could not be unambiguously met. | next: NONE

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	17212
2	preregistration	ollama_remote	glm4:latest	0	0	9639
3	novelty	ollama_remote	glm4:latest	0	0	10678
4	novelty	ollama_remote	glm4:latest	0	0	10086
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	51417
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	9053
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10010
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	14663
9	judge	ollama_remote	glm4:latest	0	0	86336

Totals: 0 input tokens, 0 output tokens, ~$0.0000 cost, 219095 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in `research/audit/da623503e2fc.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/da623503e2fc.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)
Replay tarball: `research/replay/da623503e2fc.tar.gz` (if generated)